

清华大学本科生考试试题专页纸	
考试课程：操作系统（A）	时间：2025年04月14日上午 09:50~12:15

系别： _____	班级： _____	学号： _____	姓名： _____
--------------	--------------	--------------	--------------

答卷注意 事项：	1. 答题前，请先在试题纸和答卷本上写明A卷或B卷、系别、班级、学号和姓名。
	2. 在答卷本上答题时，要写明题号，不必抄题。
	3. 试卷有20道判断题、5道问答题。
	4. 请回答所有题目。本试卷共9页。最后空白页可做草稿。
	5. 考试完毕，必须将试题纸和答卷本一起交回。

一、(20分) 判断题

- 1. [] 微内核架构的操作系统将尽量多的功能放在用户态运行，以减少内核的复杂度和提高系统的可靠性。
- 2. [] 在操作系统中，采用分页内存管理方式时，每个进程的页表必须完全加载到物理内存中才能执行该进程。
- 3. [] 在一个使用单级页表的系统中，增加 TLB的大小通常会减少平均内存访问时间。
- 4. [] 在RISC-V架构中，用户模式（U-mode）、监督模式（S-mode）和机器模式（M-mode）的特权级依次从低到高。
- 5. [] 在RISC-V架构中进行进程切换时，处理器需要先从用户模式（U-mode）进入监督模式（S-mode），然后操作系统内核保存当前进程的上下文，并恢复下一个进程的上下文。
- 6. [] 在RISC-V架构中，页表项中存储了虚地址所对应的64位物理地址。
- 7. [] 虚拟存储的主要目的是扩展物理内存的容量，使系统能够运行超过物理内存大小的程序。
- 8. [] 在RISC-V架构中，页表的根地址存储在satp寄存器中，只有在机器模式（M-mode）下才能访问该寄存器。
- 9. [] 系统调用是用户态进程请求内核提供服务的唯一方式。
- 10. [] RISC-V的中断向量表必须从物理地址0x0开始。
- 11. [] RISC-V的S模式（Supervisor mode）下可以执行所有机器指令。
- 12. [] 使用多级页表的主要目的是减少物理内存占用。
- 13. [] 反向页表（Inverted Page Table）通过虚拟地址等进行哈希计算来查找物理页框号。
- 14. [] RISC-V的页表项（PTE）中，Dirty位由硬件自动置位，无需软件干预。
- 15. [] 时间片轮转调度算法在单核RISC-V处理器上只能实现并发执行，但无法实现真正的并行执行。

16. [] 进程的“就绪→运行”状态转换仅由调度器直接触发。
17. [] 系统调用过程中，用户态参数必须通过寄存器传递，内核无法直接访问用户栈。
18. [] 进程的地址空间是固定大小的，在进程创建时就确定，运行过程中不能改变。
19. [] 中断和异常在本质上是相同的，都是由外部事件触发的对正常程序执行流程的改变。
20. [] 操作系统中，地址空间隔离可以保证不同进程之间不会相互干扰，即使某个进程发生内存访问越界也不会影响其他进程。

二、(80分)共8道问答题

1. (30分)

1.1 假设一个64位计算机系统的物理内存为8GB，页大小为4KB，页表为二级结构，每个页表项大小为8字节。请计算并给出简明计算过程与描述：一个二级结构的页表可映射的内存大小是多少？

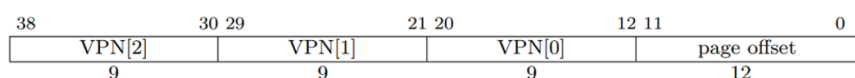
1.2 小W在编程中观察到，大多数程序在地址空间的开头部分使用大部分数据。因此，他决定设计一种新型的二级结构的页表，并将这种页表设计称为Inode Page Tables。

Inode Page Tables本质上是两级页表，但有一个不同之处：根（一级）页表前半部分的页表项（PTEs）直接映射物理页，而后半部分的页表项正常地映射到二级页表。

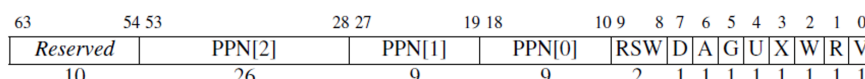
我们称前半部分的页表项对应的虚拟页称为“快页”，后半部分的页表项对应的虚拟页称为“慢页”。

假设地址为32位，页面大小为4KB，并且Inode Page Tables的根（一级）页表和二级页表都能放入单个页面，每个页表项大小为4字节，请回答以下问题：

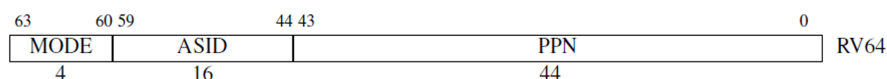
- a) 有多少个虚拟页是“快页”？
- b) 有多少个虚拟页是“慢页”？
- c) 地址空间的最大大小是多少字节？
- d) Inode Page Tables通过减少一次内存读取操作来加快“快页”的查找速度。你认为这是一种有效的优化手段吗？请简要解释。



RV64 Sv39虚地址



一个RV64 Sv39页表项（PTE）



SATP CSR寄存器格式

1.3 在RISC-V架构的Sv39分页模式（如上图所示）下，虚拟地址为39位，物理地址56位，页表为三级结构，页表项大小为8字节。请计算：a) 假设页大小为4KB，SATP寄存器所指向的根（一级）页表的页表项数量。b) 理论上每个进程的最大虚拟地址空间大小。c) 对于Sv39分页模式，除了4KB的页大小，可设置的页大小还有哪些？

1.4 小Z在学习中思考，如要构建能进行有效虚地址到物理地址映射的页表，最少所需的4KB大小的物理页面数量。于是他就根据RISC-V的Sv39分页方式，构造了如下的一个4KB的物理页面。SATP寄存器的值：0x8000 0000 0000 0040，其中最高四位的“8”表示采用Sv39分页模式。

该物理页面的内容如下

phy_addr	Value
0x00040000	0x00004001
0x00040008	0x00004001
0x00040010	0x0000400f

该物理页面中其余未表示的地方均为0

小Z认为只用这一个物理页面就可以实现Sv39的三级页表访问。如果你认为小Z的观点是错误的，请给出理由。如果你认为小Z的观点是对的，请给出这个页表所能表示的可读写的虚拟地址范围。

2. (10分)

已知 uCore 与 rCore 中的 ch3 中，定义栈的部分代码如下所示（PAGE_SIZE等常量已替换为实际数值），请同学阅读自己选择的实验的代码然后作答。

a) 一些同学在编写代码时，在多个函数中多次反复使用诸如 `int x[512]` 的大数组作为函数参数和局部变量。在测试时，发现另外一些函数出现了不可能的返回值，似乎被无缘无故篡改了。请尝试分析这个问题可能是如何导致的。

b) Rust语言以内存安全著称，但也会出现上述问题。事实上，换成C++/Go等语言写内核也会有同样的情况。为什么编程语言的编译器无法检查上述问题？

c) 该问题在现代操作系统中较少发生。内核可采用哪种机制可减少上述问题的发生？这种机制的基本思路是什么？在 RISC-V 架构上，这种机制的实现是否依赖CSR硬件寄存器？如有依赖，请说明所依赖的CSR硬件寄存器的名称和对应的功能。

```
// =====ucore=====

struct proc pool[16];

char kstack[16][4096];

__attribute__((aligned(4096))) char ustack[16][4096];

__attribute__((aligned(4096))) char trapframe[16][4096];
```

```
// =====rcore=====

#[repr(align(4096))]

#[derive(Copy, Clone)]

struct KernelStack {

    data: [u8; 8192],

}

#[repr(align(4096))]

#[derive(Copy, Clone)]

struct UserStack {

    data: [u8; 4096],

}

static KERNEL_STACK: [KernelStack; 16] = [KernelStack {

    data: [0; 8192],

}; 16];

static USER_STACK: [UserStack; 16] = [UserStack {

    data: [0; 4096],

}; 16];
```

3. (15分)

- a) 请简明扼要地描述OPT页面置换算法，并解释OPT算法在实际计算机系统中是否能实现。b) 给定页面访问序列：bcbaebdec d，物理块数为3，初始内容为空，计算OPT算法下的缺页次数与置换次数。
- b) 请简明扼要地描述工作集页面置换算法。定义工作集窗口大小 $\Delta=4$ ，给定访问序列edacdbcecead，分配了5个物理页帧，初始物理页帧为空。t从0开始，计算t=5时的工作集。请问在哪些时刻会产生缺页异常，缺页异常次数总共是多少？
- c) 请简明扼要地描述缺页率页面置换算法。如采用缺页率置换算法，容忍的缺页窗口 $T=2$ ，给定访问序列edacdbcecead，分配了5个物理页帧，初始物理页帧为空。请问在哪些时刻会产生缺页异常，缺页异常次数总共是多少？
- d) 请说明什么是Belady现象？为何FIFO页面置换算法会出现Belady现象，而LRU页面置换算法不会出现Belady现象？请判断Clock页面置换算法和LFU页面置换算法是否会出现Belady现象，并说明理由。

4. (15分)

假设某RISC-V系统运行在S模式（Supervisor mode），用户程序通过ecall触发系统调用。

假定fork函数、wait函数和printf函数能正确执行并正常返回。i) 请问如下程序执行完毕后，一共产生多少进程（最开始执行main()的进程也计入）？ii) printf 执行了多少次？iii) 如下程序从执行到结束，至少会发出几种系统调用？iv) 程序输出的前4个数字的可能序列是？并解释原因。v) main函数执行return 0后，程序是直接退出还是返回到调用main的函数继续执行？请解释原因。

```
#include <stdio.h>

void main() {
    for(int i = 1; i <=4; i++) {
        fork();
        wait(-1, 0);
        printf("%d", i);
    }
    return 0;
}
```

- fork() 表示派生一个子进程，对子进程返回 0，对当前进程返回子进程的 PID。
- wait(-1, 0) 表示当前进程进入休眠，等待任意一个子进程结束时，该调用才返回结束的子进程的 PID。如果当前进程没有子进程，则直接返回 -1。

5. (10分)

```
#include <stdio.h>

int main() {
    printf("1 / 0 = %d", 1 / 0);
    return 0;
}
```

- a) 上面是一个可在硬件环境为RISC-V64的Linux下执行的C程序，其中包含一个除以零的整数运算操作。请描述该程序在硬件环境为RISC-V64的Linux上执行的输出结果，并简要说明原因。

RISC-V 64的异常类型列表

异常类型（中文）	异常类型（英文）	触发条件
指令地址不对齐	Instruction Address Misaligned	取指令地址未按 2/4 字节对齐（如 <code>jal</code> 跳转到奇数地址）
指令访问错误	Instruction Access Fault	访问无效或权限不足的指令地址（如受保护内存）
非法指令	Illegal Instruction	译码阶段检测到无效操作码或非法 CSR 访问
断点异常	Breakpoint	执行 <code>ebreak</code> 指令或硬件断点触发
加载地址不对齐	Load Address Misaligned	加载指令（如 <code>lw</code> ）地址未按数据宽度对齐
加载访问错误	Load Access Fault	加载操作访问无效地址或权限不足
存储/原子操作地址不对齐	Store/AMO Address Misaligned	存储或原子指令地址未对齐（如 <code>sw</code> 地址非 4 字节对齐）
存储/原子操作访问错误	Store/AMO Access Fault	存储或原子操作访问无效地址或权限不足
用户模式环境调用	Environment Call from U-mode	用户模式下执行 <code>ecall</code> 指令（用于系统调用）
监管模式环境调用	Environment Call from S-mode	监管模式（S 模式）下执行 <code>ecall</code>
机器模式环境调用	Environment Call from M-mode	机器模式（M 模式）下执行 <code>ecall</code>
指令页错误	Instruction Page Fault	指令虚拟地址转换失败（页缺失或权限错误）
加载页错误	Load Page Fault	加载操作虚拟地址转换失败
存储/原子操作页错误	Store/AMO Page Fault	存储或原子操作虚拟地址转换失败

下面是ch3“多道程序与分时多任务”进行任务上下文切换的uCore/rCore参考代码：

```

1.  .globl  swtch
2.  swtch:
3.  sd ra,  0(a0)
4.  sd sp,  8(a0)
5.  sd s0,  16(a0)
6.  # ... save s1-s10
7.  sd s11, 104(a0)
8.
9.  ld ra,  0(a1)
10. ld sp,  8(a1)
11. ld s0,  16(a1)
12. # ... restore s1-s10
13. ld s11, 104(a1)14. ret

```

程序片段一：uCore中的switch汇编函数

```
1. .altmacro
2. .macro SAVE_SN n
3. sd s\n, (\n+2)*8(a0)
4. .endm
5. .macro LOAD_SN n
6. ld s\n, (\n+2)*8(a1)
7. .endm
8. .section .text
9. .globl switch
10. switch: sd sp, 8(a0) sd ra, 0(a0)
11. .set n, 0
12. .rept 12 SAVE_SN %n
13. .set n, n + 1
14. .endr
15. ld ra, 0(a1)
16. .set n, 0
17. .rept 12 LOAD_SN %n
18. .set n, n + 1
19. .endr
20. ld sp, 8(a1)
21. ret
```

程序片段二：rCore中的switch汇编函数

b) 请根据你对你完成的实验的理解，解释函数上下文切换、任务上下文切换、Trap上下文切换的含义，并进一步说明三者的区别是啥？

c) 请说明上述代码中，是否存在特权指令？如有，请具体指出特权指令在第几行。

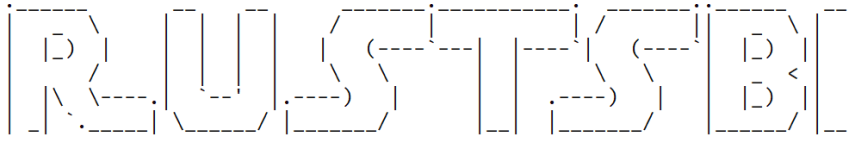
在实验环境搭建过程中，通过QEMU软件来模拟一台计算机，并会执行如下的命令行脚本，来让QEMU模拟加电启动虚拟计算机，运行操作系统和应用：

```
qemu-system-riscv64 -machine virt -nographic -bios ../bootloader/rustsbi-qemu.bin \
-device loader,file=target/riscv64gc-unknown-none-elf/release/os.bin,addr=0x80200000
```

或

```
qemu-system-riscv64 -nographic -machine virt -bios ../bootloader/rustsbi-qemu.bin \
-kernel build/kernel
```

并可得到类似如下内容：



```
[rustsbi] Implementation: RustSBI-QEMU Version 0.0.2
[rustsbi-dtb] Hart count: cluster0 with 1 cores
[rustsbi] misa: RV64ACDFIMSU
[rustsbi] mideleg: ssoft, stimer, sext (0x222)
[rustsbi] medeleg: ima, ia, bkpt, la, sa, uecall, ipage, lpage, spage (0xb1ab)
[rustsbi] pmp0: 0x10000000 ..= 0x10001fff (rwx)
[rustsbi] pmp1: 0x80000000 ..= 0x8fffffff (rwx)
[rustsbi] pmp2: 0x0 ..= 0xfffffffffffff (---)
qemu-system-riscv64: clint: invalid write: 00000004
[rustsbi] enter supervisor 0x80200000
Hello, world!
.text [0x80200000, 0x80202000)
```

- d) 请解释在上面的两种命令行脚本中QEMU的各参数的含义。基于这两种脚本的QEMU模拟加电启动过程，分别把哪些内容加载到它模拟的物理内存中了？这两种脚本在具体执行中，有何不同之处？
- e) QEMU从加电启动到执行操作系统第一条指令的时间段中，是否出现了特权级切换？是否出现了地址空间切换？是否出现了任务切换？
- f) 解释“Hello, world”的输出是如何成功执行的？

草稿页